Python Assignment

Write at least five code examples

This section will cover the following topics:

- Python functions
- Lambda functions (Syntax is lambda arguments: expression)
- Numpy
- Pandas and Reading CSV files
- If Statements
- Lists, Tuples, Sets, Dictionaries
- Python String Methods

Python Functions

In [68]:
```
#Python functions

def say():
    return "Hi Siddhant"
print(say())
```

```
Hi Siddhant
```

In [66]:
```
def square(x):
    return x * x  # * stands for ^2.
print(square(4))
```

```
16
```

In [64]:
```
def product (x):
    return x ** x  # * is x^2 and ** means x^4.
print(product(4))
```

```
256
```

In [62]:
```
x=10
def sum():
    y=5
    return x + y
print(sum())
```

```
15
```

In [60]:
```
#Argument value

def say(name="Guest"):
    return f"Hi, {name}!" #f-string used here and
```

```
print(say("Siddhant"))
print(say())
```

```
Hi, Siddhant!
Hi, Guest!
```

In [98]:
```
#Recursive Functions

def num(n):
    if n==0:
        return 1
    else:
        return n * num(n-1) # 4 * 3 * 2 * 1
print(num(4))
```

```
24
```

In [100…
```
def product(a,b):
    """Multiplies two numbers a and b.""" # use of docstring(documentation string)
    return a * b
print(product(4,5))
print(product.__doc__)
```

```
20
Multiplies two numbers a and b.
```

In [116…
```
def power (a,b):
    """calculates a^b""" ## use of docstring
    return a ** b
print(power(2,3))
print(power.__doc__)
```

```
8
calculates a^b
```

Lambda functions

A lambda function is a small anonymous function. A lambda function can take any number of arguments, but can only have one expression. Anonymous: For eg, without def.

In [105…
```
# simple lambda function
addfive = lambda x:x+5
addfive(10)
```

Out[105… 15

In [109…
```
sum= lambda x,y: x + y
print(sum(3,5))
```

```
8
```

In [113…
```
#Comparing lambda functions with regular functions

# above code statement without lambda

def sum(x,y): #function named sum
```

```
    return x + y
print(sum(3,5))
```

```
8
```

In [111…

```
#Comparing lambda functions with regular functions

# above code statement without lambda

x=3  # no function involved.
y=5
sum= x + y
print(sum)
```

```
8
```

In [120…

```
# map function in lambda

myvalues = [1,2,3,4,5,6,7,8,9,10]
newvalues = list(map(lambda x:x+5, myvalues))

""" list is a collection of items (like numbers, strings, etc.) and in your code, m
The map() function applies a specific operation to each item in a list (or any iter

print(newvalues)
```

```
[6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

In [124…

```
#uisng filter function to get even numbers

myvalues = [1,2,3,4,5,6,7,8,9,10]
newvalues = list(filter(lambda x:x%2 == 0, myvalues))

""" lambda x: x % 2 == 0 defines a small anonymous function that checks if x is eve
It returns True, if the remainder when x is divided by 2 is 0.
If the function returns True, that element will be included in the result; if False

print(newvalues)
```

```
[2, 4, 6, 8, 10]
```

In [128…

```
#using redue function to get product of all numbers

from functools import reduce

myvalues = [1,2,3,4]
newvalues = reduce(lambda x,y: x * y, myvalues)

""" Not using list here because reduce is designed to work with any iterable (like

print(newvalues)
```

```
24
```

NumPy

NumPy (Numerical Python) is a library (collections of code and functions) in Python used for numerical computing. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays.

In [133…
```
import numpy as np
```

In [165…
```
#Create different types of NumPy arrays (1D, 2D, 3D).

"""An array is a data structure that can hold multiple values of the same type in a
 it allows to store several pieces of data (values) together, all organized under o

#1D array

"""A 1D array (one-dimensional array) that stores a collection of items in a single

x= np.array([1,2,3,4,5])
print("1D Array:\n", x)  #"1D Array:\n" is a string literal that uses to format you
                         #here, \n is a special character that represents a newline


print(x[0])             #outputs the 1st element of array using indexing
```

```
1D Array:
 [1 2 3 4 5]
1
```

In [171…
```
#2D array

""" A 2D array  organizes data in a grid format, with rows and columns.
    For this, output gives 2 rows and 3 columns"""

x= np.array([[1,2,3], [4,5,6]])
print("\n2D Array: (Matrix)\n", x)  #Matrix just refers the array can be used in ma
```

```
2D Array: (Matrix)
 [[1 2 3]
 [4 5 6]]
```

In [173…
```
#3D array

""" A 3D array extends the concept of a 2D array into an additional dimension."""

x= np.array([[[1,2,3],[4,5,6]], [[7,8,9],[10,11,12]]])
print("\n3D Array (3D Matrix):\n", x)                    #"3D matrix" only emphasi
```

```
3D Array (3D Matrix):
 [[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]]
```

In [183…
```
#Basic Arithmetic Operations on Arrays

a= np.array([1, 2, 3, 4])
b= np.array([10, 20, 30, 40])
```

```
print("sum:\n", a + b) #
```

```
sum:
 [11 22 33 44]
```

In [187…
```
# Indexing and Slicing to Access Elements

a= np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print("Element at (1,2):", a[1,2])
```

```
Element at (1,2): 6
```

In [209…
```
#slicing Subarray with first two rows and first two columns

""" Slicing in NumPy allows you to extract a portion (subarray) of an array using a

a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
subarray = a[:2, :2]

""" Slicing index is a[start:end].
(:2) means take rows from the start up to (but not including) index 2."""

print("\nSubarray:\n", subarray)
```

```
Subarray:
 [[1 2]
 [4 5]]
```

In [215…
```
a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

""" The index 0 refers to the first row of the matrix."""

print("\nFirst row:", a[0])
```

```
First row: [1 2 3]
```

In [217…
```
a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

"""The colon : before the comma means "select all rows".
The 1 after the comma means "select the second column"."""

print("\nSecond column:", a[:, 1])
```

```
Second column: [2 5 8]
```

In [225…
```
# using reshape method of Numpy array from 1D into a 2D array (2 rows, 3 columns)

a= np.array([1, 2, 3, 4, 5, 6])
reshaped= a.reshape(2,3)      #reshaped is simply a variable that holds the result

print("2D array:\n", reshaped)
```

```
2D array:
 [[1 2 3]
 [4 5 6]]
```

In [235…
```
## swapping the rows and columns of the reshaped array using Transpose

a= np.array([1, 2, 3, 4, 5, 6])

reshaped= a.reshape(2,3)
transposed = reshaped.T  # .T attribute is a shorthand for transposing the array.
                         # transposed is a variable.

""" The first row [1, 2, 3] becomes the first column [1, 4].
    The second row [4, 5, 6] becomes the second column [2, 5].
    Each element in the original rows moves to the corresponding column in the tran

print("\nTransposed Array:\n", transposed)
```

```
Transposed Array:
 [[1 4]
 [2 5]
 [3 6]]
```

In [241…
```
#Create and Use NumPy Random Number Generators

random_array = np.random.rand(5)  #this line generates an array of 5 random floatin

""" np.random.rand(5) does
    1) creates creates an array filled with random numbers between 0 and 1.
    2) The argument 5 specifies that you want 5 random numbers."""

print("Random Array:", random_array)
```

```
Random Array: [0.40535279 0.61109246 0.13770549 0.21058619 0.06806957]
```

Pandas and Reading CSV files

DataFrame: It is a two-dimensional, table-like data structure used to orgainse data, data analyis, Handling Large Datasets. Also, it allows to access data easily using labels (like column names) instead of relying on numerical indices.

In [245…
```
import pandas as pd
```

In [268…
```
#Create DataFrame on CSV file

# Load your CSV file into a Pandas DataFrame
file_path = file_path = r"C:\Users\priya\Desktop\Sid\Chaitrali\New folder\PR\Siddha

"""Used raw strings above by adding an r before the string.
   So, it treats backslashes as literal characters and not as escape sequences."""

Customers_df = pd.read_csv(file_path)
```

In [265…
```
# Display the first few rows of the DataFrame

print(Customers_df.head(2))
```

```
   CustomerID FirstName LastName                  Email   BirthDate
0           1      John      Doe  johndoe22@example.com  1990-01-01
1           2    Aniket  Gaikwad   aniket14@example.com  1994-01-01
```

In [276…

```
#For Data Cleaning, checking missing data

print("\nMissing Values:\n", Customers_df.isnull().sum())
```

```
Missing Values:
 CustomerID    0
FirstName     0
LastName      0
Email         0
BirthDate     0
dtype: int64
```

In [282…

```
#Data Manipulation Tasks by Renamed 2nd and 3rd columns


Customers_df.rename(columns={"FirstName": "First Name", "LastName": "Last Name"}, i
print("\nRenamed Columns:\n", Customers_df.head())
```

```
Renamed Columns:
    CustomerID First Name Last Name                  Email   BirthDate
0           1       John       Doe   johndoe22@example.com  1990-01-01
1           2     Aniket   Gaikwad    aniket14@example.com  1994-01-01
2           3      Mayur  Lasurkar     mayur13@example.com  1995-01-21
3           4        Sid     Patil       sid11@example.com  1995-01-11
4           5   Priyanka    Thatha  priyanka21@example.com  1993-01-01
```

In [292…

```
#Explore Data Analysis and Visualization Using Pandas

# Analyze the distribution of ages (assume you have a column 'BirthDate')

Customers_df['Age'] = 2024 - pd.to_datetime(Customers_df['BirthDate']).dt.year  #ne

"""pd.to_datetime(Customers_df['BirthDate']): This converts 'BirthDate' column, whi
                                    .dt.year: This extracts the year part from the


# Calculate age
print("\nAge Data:\n", Customers_df['Age'].describe())  #Customers_df['Age'].descri
```

```
Age Data:
 count     5.000000
mean     30.600000
std       2.073644
min      29.000000
25%      29.000000
50%      30.000000
75%      31.000000
max      34.000000
Name: Age, dtype: float64
```

In [306…

```
#shows the unique last names with their counts, i.e. by grouping the DataFrame by t
```

```
grouped_data = Customers_df.groupby('Last Name').size()
print("\nCx's Last Name:\n", grouped_data)
```

```
Cx's Last Name:
 Last Name
Doe         1
Gaikwad     1
Lasurkar    1
Patil       1
Thatha      1
dtype: int64
```

In [304…

```
#Create Pivot Table show counts of customers grouped by last names

pivot_table = Customers_df.pivot_table(index='Last Name', values='CustomerID', aggf
print("\nPivot Table:\n", pivot_table)
```

```
Pivot Table:
            CustomerID
Last Name
Doe                 1
Gaikwad             1
Lasurkar            1
Patil               1
Thatha              1
```

If Statements

In [316…

```
#using if Statement
weight = 75

if weight > 70:
    print("Not qualified for Olympics")
```

```
Not qualified for Olympics
```

In [322…

```
#using if-else statement
weight = 70

if weight <= 70:
    print("Qualified for Olympics")
else:
    print("Qualified for Asian Games")
```

```
Qualified for Olympics
```

In [330…

```
##using if-elif-else statement

weight = 62

if weight == 70:
    print("Qualified for Olympics")

elif weight < 65:
    print("Qualifed for domestic games.")
```

```
else:
    print("Qualified for Asian Games")
```

```
Qualifed for domestic games.
```

In [332…
```
# using conditional expressions.
condition = input("Enter a number between 1 and 10. ")
val = int(condition)
if val ==5:
    print("Your number is 5")
```

```
Your number is 5
```

In [334…
```
#using Nested if statements
score = 85

if score >= 60:
    print("You passed the exam!")

    if score >= 90:                 #Nested if for Higher Scores: to check if the s
        print("Excellent job!")
    elif score >= 75:
        print("Good job!")
    else:
        print("You can do better next time.")
else:
    print("You failed the exam.")
```

```
You passed the exam!
Good job!
```

Loops

- For loop is used to repeat a block of code a specific number of times. It will run the loop (like going through a list).
- A while loop is used to repeat a block of code as long as a certain condition is true. The loop continues until a condition changes

In [339…
```
#Using for loops to iterate over sequences

games_list = ['Cricket', 'Football', 'Hockey']

for game in games_list:
    print(game)
```

```
Cricket
Football
Hockey
```

In [345…
```
#Using while loops for indefinite iteration

count = 0                           #starting counting from zero.

while count < 5:                    #loop will continue until count is not less t
```

```
    print("Count:", count)
    count += 1                          #value of count increase by 1 each time the l
```

```
Count: 0
Count: 1
Count: 2
Count: 3
Count: 4
```

In [347…

```
# Nested loops for a multiplication table from 1 to 3

for i in range(1, 4):                   # Outer loop for the first number (1 to 3)
    for j in range(1, 4):               # Inner loop for the second number (1 to 3
        product = i * j
        print(f"{i} x {j} = {product}")  # prints the multiplication statement usin
    print()                             # Print a blank line after each row
```

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3

2 x 1 = 2
2 x 2 = 4
2 x 3 = 6

3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
```

In [351…

```
#using break:

somestring= "1020x954D5837"
for i in somestring:
    print(i)
    if i == "x":
        break
```

```
1
0
2
0
x
```

In [353…

```
#using continue

for num in range(5):
    if num == 2:
        continue  # Skip the rest of the code when num equals 2
    print(num)
```

```
0
1
3
4
```

Lists, Tuples, Sets, Dictionaries

- Tuples are immutable, so we can't change elements directly. (Immutable, ordered, allows duplicates)
- Lists are flexible and can be modified, while tuples are fixed and cannot be changed. (Mutable, ordered, allows duplicates)

  *** Use lists when collection need to be change, and use tuples when requires a stable, unchangeable collection of items.***

- A set is an unordered collection of unique elements. It does not allow duplicates. (Unordered, does not allow duplicates)
- A dictionary is a collection of key-value pairs. Each key must be unique, and values can be of any type. (Key-value pairs, unordered, keys must be unique)

In [406…
```
#Create and Manipulate Lists

games = ['Cricket', 'Football', 'Hockey']

# Accessing elements using indexing
print(games[0])
```

```
Cricket
```

In [410…
```
#Modifying elements

games[1] = 'basketball'
print(games)
```

```
['Cricket', 'basketball', 'Hockey']
```

In [412…
```
# Adding elements

games.append('Swimming')    #append is a built-in method for Python lists
print(games)
```

```
['Cricket', 'basketball', 'Hockey', 'Swimming']
```

In [365…
```
#Create and Manipulate Tuples
games = ('Cricket', 'Football', 'Hockey')

# Accessing elements using indexing
print(games[1])
```

```
Football
```

In [368…
```
#concatenate tuples to create a new one.

new_games = games + ('swimming',)
print(new_games)
```

```
('Cricket', 'Football', 'Hockey', 'swimming')
```

In [373…
```
#Create and Manipulate Sets

games = {'Cricket', 'Football', 'Hockey'}
```

```
# Adding elements
games.add('Swimming')
print(games)
```

```
{'Football', 'Hockey', 'Cricket', 'Swimming'}
```

In [400…
```
#Create Dictionaries

x = {
    'name': 'Siddhant',
    'age': 29,
    'email': 'sid@example.com'
}
```

In [378…
```
## Accessing elements using keys

print(x['name'])
```

```
Siddhant
```

In [380…
```
# Adding new key-value pair

x['city'] = 'Toronto'
print(x)
```

```
{'name': 'Siddhant', 'age': 29, 'email': 'sid@example.com', 'city': 'Toronto'}
```

Operators

In [386…
```
#Arithmetic Operators
a = 10
b = 3

# Addition
print(a + b)
```

```
13
```

In [390…
```
# Example 2: Comparison Operators
x = 7
y = 5

# Greater than
print(x > y)
```

```
True
```

In [392…
```
#Logical Operators
a = 4
b = 6

print(a > 2 and b < 10)
```

```
True
```

In [394…
```
#Assignment Operators
x = 10
print(x)
```

```
10
```

In [398…
```
#Add and assign

x += 5
print(x)
```

```
20
```

In [416…
```
#Operator Precedence

result = 2 + 3 * 4
print(result)
```

```
14
```

Python String Methods

In [431…
```
#Concatenation and Slicing

say = "Hello"
who = "Everyone"
```

In [433…
```
# Concatenation (joining two strings)

say2 = say + ", " + who + "!"

print(say2)
```

```
Hello, Everyone!
```

In [435…
```
#Uppercase, Lowercase, and Title Case

sentence = "to be or not to be"

# Convert to uppercase
upper_sentence = sentence.upper()
print(upper_sentence)
```

```
TO BE OR NOT TO BE
```

In [437…
```
# Convert to lowercase

lower_sentence = sentence.lower()
print(lower_sentence)
```

```
to be or not to be
```

In [443…
```
# Convert to title case

title_sentence = sentence.title()
print(title_sentence)
```

```
To Be Or Not To Be
```

In [445…
```
#Removing Whitespace
sentence = "   Good, Morning!   "

#Remove spaces from both sides
new_sentence = sentence.strip()
print(new_sentence)
```

```
Good, Morning!
```

In [459…
```
# Remove leading spaces (left side)

line = "   Good, Morning     !   "

left_line = line.lstrip()
print(left_line)
```

```
Good, Morning     !
```

In [461…
```
# Remove trailing spaces (right side)

line = "   Good, Morning     !   "

right_line = line.rstrip()
print(right_line)
```

```
  Good, Morning     !
```

In [465…
```
#Splitting Strings
games = "Cricket, Football, Hockey"

# Split the string using comma as the separator

games = games.split(", ")
print(games)
```

```
['Cricket', 'Football', 'Hockey']
```

In [471…
```
# String Replace and Length

text = "Good Morning"

# Replace 'Morning' with 'Evening'
new_text = text.replace("Morning", "Evening")
print(new_text)

# Find the length of the string
length = len(text)
print(length)
```

```
Good Evening
12
```

In [ ]: